

Lecture 5: Parallelization and UW Resources

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Understand what parallel computing is and the basics of how it works
2. Learn to use multithreading and multiprocessing in Julia
3. Get set up on UW-Madison's computing resources (SSCC)

Parallel Computing Concepts

Parallelization

- Central Processing Unit (CPU): executes the instructions of a computer program
- Modern CPUs have multiple **cores**
 - Different cores can execute different tasks at the same time
- Two main models of parallelism in Julia:
 - **Multithreading**: several threads within one Julia process, sharing memory
 - **Multiprocessing**: several independent Julia processes, each with its own memory
- Intuition:
 - Multithreading: multiple people (threads) baking pizza (executing code) in the *same* kitchen (core / memory)
 - Multiprocessing: multiple people baking pizza in *separate* kitchens (cores)

Multiprocessing or multithreading?

- Multithreading:
 - All workers share the same memory
 - Fast setup, no copying of data across workers
 - Need to be careful when workers write to the same memory (**data race**)
 - All workers must be on the same machine
- Multiprocessing:
 - All workers have their own memory (kitchen, ingredients)
 - Slower to set up and to share data
 - Need to be explicit about what each worker knows (@everywhere)
 - Workers can span multiple machines/servers
- Rule of thumb: start with **multithreading** for loops over grid points or simulations on one machine; use multiprocessing for big jobs across servers

Multithreading in Julia

- Start Julia with threads: `julia --threads=8` (or set it in the VS Code Julia extension settings)

```
Threads.nthreads()           # how many threads do I have?

Threads.@threads for i in 1:nk
    V_new[i] = bellman_at_k(p, V, i)  # each i independent: safe
end

# DANGER: data race (threads write to the same variable)
total = 0.0
Threads.@threads for i in 1:N
    total += x[i]                    # WRONG: threads clobber each other
end
```

- Safe pattern: write to *different* slots of a pre-allocated array, then combine
- Debugging tip: if the bug disappears without `@threads`, you likely have a race

Multiprocessing in Julia

```
using Distributed
addprocs(4)                # spin up 4 worker processes

@everywhere using Optim    # workers need their own packages
@everywhere f(x) = expensive_computation(x)

results = pmap(f, 1:100)   # distribute tasks across workers

# SharedArrays: one array all workers on a machine can write to
using SharedArrays
V = SharedArray{Float64}(nk)
@sync @distributed for i in 1:nk
    V[i] = bellman_at_k(i)
end
```

- pmap is best for a moderate number of expensive tasks; @distributed for many cheap ones

Example: solving and simulating a savings model

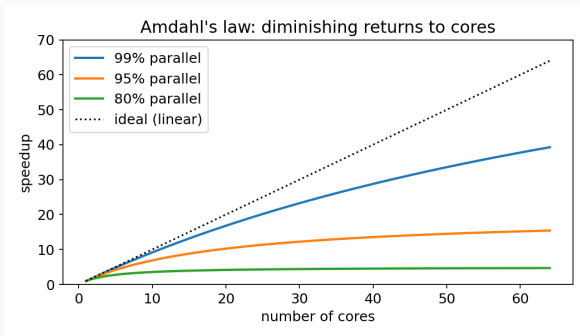
- Value function iteration to solve for V :

$$V_{n+1}(z, k) = \max_{c, k'} \{ \log(c) + \beta E_{z'} [V_n(z', k') \mid z] \}, \quad c + k' = zk^\alpha + (1 - \delta)k$$

- Solving on a capital grid $\{k_1, \dots, k_N\}$: different cores solve the maximization at different k points, then we gather results and iterate
- After solving: simulate many histories of $\{c_t, k_t\}$ given random productivity paths $\{z_t\}$ — each simulated path is independent, so this parallelizes perfectly (“embarrassingly parallel”)

No free lunch

- Parallelization is not a free lunch because of overhead costs
 - Spinning up workers and sharing memory takes time
 - More cores running means more memory in use
- Diminishing returns kick in quickly for small problems (Amdahl's law: the serial fraction of your code bounds the possible speedup)



UW-Madison Computing Resources

Social Science Computing Cooperative (SSCC)

- <https://sscc.wisc.edu> — check the current docs; hardware specs change year to year
 1. **Linstat**: interactive Linux servers with dozens of cores and hundreds of GB of memory
 - Good for interactive work and for checking that your code runs on SSCC servers before submitting to Slurm
 2. **Slurm**: a cluster of servers with thousands of cores in total
 - Submit batch jobs that can run for days
- Be mindful that other people need to use Linstat and Slurm, too
- SSCC has other resources for jobs that need lots of memory and for confidential data (SILO)
- Open OnDemand gives you an interactive desktop/JupyterLab in the browser, running on SSCC hardware

Center for High Throughput Computing (CHTC)

- <https://chtc.cs.wisc.edu/> — campus-wide resource, separate from SSCC
- High Performance Computing (HPC) cluster: large multi-core/multi-node jobs
- High Throughput Computing (HTC): many independent smaller jobs
- Free for UW research; worth knowing it exists if your problem outgrows the SSCC

Accessing Linstat and Slurm

- You should have an SSCC account from first-year orientation (if not: request one on the SSCC website)
- Options for connecting:
 - SSH directly: `ssh username@linstat.ssc.wisc.edu` (use the SSCC VPN off campus)
 - Open OnDemand in the browser (interactive)
 - Winstat (Windows remote desktop) if you prefer a GUI
- VS Code's **Remote-SSH** extension lets you edit and run code on Linstat with your normal editor — highly recommended
- Your SSCC home directory is shared across Linstat, Slurm, and Winstat

Linstat cheat sheet

```
julia                # open an interactive Julia REPL
julia script.jl > output.txt & # run a script, save output, keep running
                                # after you log off (with nohup)
cd folder/           # change directory
ls                   # list files
top                  # see jobs running (q to quit)
kill PID             # stop job with id PID
```

- To use N threads on Linstat: `julia --threads=N script.jl`
- Please don't hog an entire Linstat server — check `top` before grabbing all the cores

Slurm cheat sheet

```
# submit a job with N cores and M GB of memory:
ssubmit --cores=N --mem=Mg "julia --threads=N script.jl"

squeue -a          # see jobs running on Slurm
scancel JOBID      # cancel your job
```

- To figure out how much memory you need: run the job on Linstat, check its memory use in top (RES), then kill it and submit to Slurm
- Slurm emails you when your job finishes (success *or* failure)
- If precompiled package images cause errors on Slurm, add `--pkgimages=no` to the Julia call
- Details and current limits: <https://sscc.wisc.edu/sscc/pubs/grc/>

- Linstat is good for:
 1. Interactive coding
 2. Checking that your code runs on SSCC servers
 3. Seeing how much memory you are using
 4. Sending jobs to Slurm
- Slurm is good for big jobs that need lots of cores and run for a long time
- To the code!